
Autoregressive Models for Financial Prediction: Educational Modules

Matthew Hurtado
Department of Computer Science
Department of Mathematics
Rensselaer Polytechnic Institute
Troy, NY 12180
hurtam@rpi.edu

Benjamin Rodriguez
Department of Mathematics
Rensselaer Polytechnic Institute
Troy, NY 12180
rodrib3@rpi.edu

Darwin Vargas
Department of Computer Science
Rensselaer Polytechnic Institute
Troy, NY 12180
vargad@rpi.edu

Abstract

Learning the mathematics of machine learning is a challenging task and in many cases it is difficult to connect the theory with the implementation. In this paper, a series of educational modules designed to help students understand autoregressive models, with a focus on financial market predictions are outlined. These modules are aimed at undergraduate computer science and mathematics majors; however, they remain accessible to most audiences. They cover a variety of important topics, including data preparation, implementing autoregressive models, evaluating their performance, and potential scaling problems. Students will navigate several exercises that reinforce their understanding. By the end of the modules, students will have acquired the tools to build, evaluate, and optimize predictive models in the context of financial data, understanding autoregressive models' role in the broader field of machine learning and statistical modeling.

1 Introduction

Accurate market prediction is crucial for successful trading because determining whether to buy or sell a given stock depends on its future price. However, predicting such prices is challenging because there are several relevant economic factors that contribute to the complexity of the stock market, as well as non-economic factors including geopolitical and psychological influences.

Autoregressive models, a type of time-series forecasting method, provide a framework for making predictions about future events based on historical data. They use the previous predictions, in sometimes non-linear ways, to predict the next values. This makes them ideal for forecasting, where historical patterns can influence future outcomes. In this study, financial modeling is targeted as an ideal example for autoregressive implementation; however, these modules generalize to any topic that may call for an autoregressive model.

In the Github Repository you can find supplemental material, including code and more details on the implementations. In the final section, current research on applying quantum mechanical principles to transformer-based market prediction is explored.

2 Problem Statement

The stock prediction problem is modeled as a time series regression and the prediction goal is formulated as a minimization problem.

2.1 Problem Formulation

In the context of stock market prices, each data point refers to a stock price at a particular time. For example, consider several consecutive closing prices $p_1, p_2, \dots, p_{t-1}$. Here, each p_i represents the closing price of the stock at time i . The goal of time series analysis then becomes predicting p_t based on $[p_{t-k}, \dots, p_{t-2}, p_{t-1}]$.

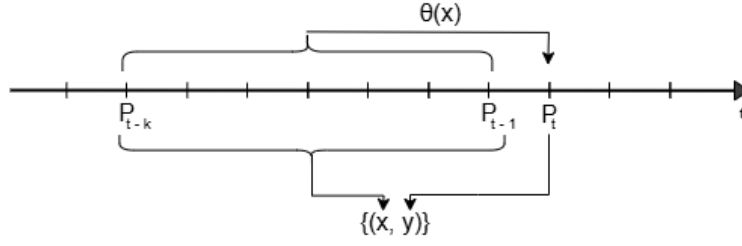


Figure 1: Time series with (x,y) pair for every **sliding/rolling window** of size k

A rolling window of size k is defined as the input for p_t and for p_{t+1} the rolling window shifts by one. Time series data differs from other data because it inherently incorporates **time dependencies**, an important property of financial modeling. A general model $\phi(\cdot)$ will output p_t based on the window of k prices as follows:

$$p_t = \phi(p_{t-k}, \dots, p_{t-1}). \quad (1)$$

It is obtained through training, where the goal is to approximate the **target function** f as closely as possible, according to a **loss function** $L(\hat{\mathbf{p}}, \mathbf{p})$.

3 Regression Models

3.1 Linear Regression

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: A linear regression model employs coefficients as follows:

$$p_t = w_0 + w_1 p_{t-k} + \dots + w_k p_{t-1} + \epsilon_t, \quad (2)$$

where $\{w_0, w_{t-k}, \dots, w_{t-1}\}$ are $k + 1$ unknown coefficients to be learned from the training dataset.

We rewrite it as follows

$$p_t = \hat{p}_t + \epsilon_t = \mathbf{w}^\top \mathbf{x}_t + \epsilon_t, \quad (3)$$

where the input time series is a vector $\mathbf{x}_t$ and the coefficients is represented as a vector $\mathbf{w}$, as follows

$$\mathbf{x}_t = \begin{bmatrix} 1 \\ p_{t-k} \\ \vdots \\ p_{t-1} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_k \end{bmatrix}. \quad (4)$$

Letting $\hat{\mathbf{p}} = \begin{bmatrix} \hat{p}_1 \\ \vdots \\ \hat{p}_n \end{bmatrix}$ and $\mathbf{p} = \hat{\mathbf{p}} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$, where n is the number of predictions being made, use an arbitrary loss function such as MSE:

$$L(\hat{\mathbf{p}}, \mathbf{p}) = \frac{1}{n} \sum_{t=1}^n (\mathbf{w}^\top \mathbf{x}_t - p_t)^2 \quad (5)$$

Model Structure: The linear regression model computes the output as $\hat{p}_t = \mathbf{w}^\top \mathbf{x}_t$, where $\mathbf{w}$ includes the bias term c as its first element, and $\mathbf{x}_t$ is augmented with a constant 1 for consistent computation.

Training Process: The model is trained by minimizing the loss function $L(\hat{\mathbf{p}}, \mathbf{p})$ over a given dataset. Gradient descent or other optimization algorithms (e.g., stochastic gradient descent or normal equation methods) are used to update the weight vector $\mathbf{w}$ iteratively:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\hat{\mathbf{p}}, \mathbf{p}), \quad (6)$$

where η is the learning rate, and $\nabla_{\mathbf{w}} L$ is the gradient of the loss with respect to $\mathbf{w}$.

Inference Process: Once the weight vector $\mathbf{w}$ is learned, predictions for new input vectors $\mathbf{x}_t$ can be made by computing:

$$\hat{p}_t = \mathbf{w}^\top \mathbf{x}_t. \quad (7)$$

Complexity: The computational complexity for training the linear regression model depends on the method used: - For gradient descent, the complexity is $O(k \cdot n \cdot \text{epochs})$, where k is the number of features (including the bias term), n is the number of training samples, and epochs is the number of iterations. - For solving the normal equations directly, the complexity is $O(k^3)$ due to the matrix inversion step.

For inference, predicting a single value requires $O(k)$ operations.

3.2 ARMA

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$

Objective Function: The ARMA model is similar to the linear regression model, however, it also incorporates its m previous errors into its prediction as follows:

$$p_t = c + \sum_{i=1}^k w_{t-i} p_{t-i} + \sum_{i=1}^m \theta_{t-i} \epsilon_{t-i} + \epsilon_t \quad (8)$$

where $\{c, \varphi_{t-k}, \dots, \varphi_{t-1}, \theta_{t-m}, \dots, \theta_{t-1}\}$ are $k + m + 1$ unknown coefficients to be learned from the training dataset. We rewrite as follows:

$$p_t = \hat{p}_t + \epsilon_t = \mathbf{w}^\top \mathbf{x}_t + \epsilon_t \quad (9)$$

where the input time series is a vector $\mathbf{x}_t$ and the coefficients is represented as a vector $\mathbf{w}$, as follows

$$\mathbf{x}_t = \begin{bmatrix} 1 \\ p_{t-k} \\ \vdots \\ p_{t-1} \\ \epsilon_{t-m} \\ \vdots \\ \epsilon_{t-1} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} c \\ w_{t-k} \\ \vdots \\ w_{t-1} \\ \theta_{t-m} \\ \vdots \\ \theta_{t-1} \end{bmatrix} \quad (10)$$

Letting $\hat{\mathbf{p}} = \begin{bmatrix} \hat{p}_1 \\ \vdots \\ \hat{p}_n \end{bmatrix}$ and $\mathbf{p} = \hat{\mathbf{p}} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$, where n is the number of predictions we are making, we use MSE as the loss function:

$$L(\hat{\mathbf{p}}, \mathbf{p}) = \frac{1}{n} \sum_{t=1}^n (\mathbf{w}^\top \mathbf{x}_t - p_t)^2 \quad (11)$$

Model Structure: The ARMA regression model computes the output as $\hat{p}_t = \mathbf{w}^\top \mathbf{x}_i$, where $\mathbf{w}$ includes the bias term c , the AR weights, and the MA (the error) weights. $\mathbf{x}_i$ is augmented with a constant 1 for consistent computation.

Training Process: The model is trained by minimizing the loss function $L(\hat{\mathbf{p}}, \mathbf{p})$ over a given dataset. Gradient descent, or other optimization algorithms, is used to update the weight vector $\mathbf{w}$ iteratively:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\hat{\mathbf{p}}, \mathbf{p}) \quad (12)$$

Inference Process: Once the weight vector $\mathbf{w}$ is learned, predictions for the new input vectors $\mathbf{x}_t$ can be made by computing:

$$\hat{p}_t = \mathbf{w}^T \mathbf{x}_t. \quad (13)$$

Complexity:

- **Training:** $O(k \cdot n \cdot \text{epochs})$
- **Inference:** $O(k)$

3.3 Decision Trees

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: For a Decision Tree, the goal is to divide the data set into two groups, G_1 and G_2 , based on a feature j and a threshold s that minimizes the variance of the two groups.

The variance for a given group G is as follows:

$$\sigma^2(G) = \frac{1}{|G|} \sum_{p_i \in G} (p_i - \mu)^2 \quad (14)$$

where μ is the mean of G . The MSE of the two groups is calculated as follows:

$$\text{MSE}(G_1, G_2) = \frac{|G_1|}{n} \cdot \sigma^2(G_1) + \frac{|G_2|}{n} \cdot \sigma^2(G_2) \quad (15)$$

The predicted value, $\hat{p}_t$, is given by the leaf node of the tree that is reached when the tree is traversed with the given input $\{p_1, p_2, \dots, p_{t-1}\}$.

Letting $\hat{\mathbf{p}} = \begin{bmatrix} \hat{p}_1 \\ \vdots \\ \hat{p}_n \end{bmatrix}$ and $\mathbf{p} = \hat{\mathbf{p}} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$, where n is the number of predictions we are making, we use MSE as the loss function:

$$L(\hat{\mathbf{p}}, \mathbf{p}) = \frac{1}{n} \sum_{t=1}^n (\hat{p}_t - p_t)^2 \quad (16)$$

Model Structure: A Decision Tree recursively splits the input space based on decisions at each internal node:

$$x_j \leq s \quad (\text{split condition}) \quad (17)$$

Each leaf node outputs a prediction $\hat{y}$.

Training Process: The tree is constructed by iteratively splitting the dataset to minimize the MSE for each split. At each node, the algorithm selects the feature x_j and threshold s that minimize:

$$\text{MSE}(G_1, G_2),$$

where G_1 and G_2 are the two resulting groups after the split. Splitting stops when a predefined stopping criterion is met, such as a maximum depth or minimum group size.

Inference Process: For a new input vector $\mathbf{x}_t$, the prediction $\hat{y}_t$ is obtained by traversing the tree from the root to a leaf, following the decision rules at each node. At each internal node v_k , the node transition is determined by the truth value of $\mathcal{R}_k(\mathbf{x}_t)$:

$$v_k \xrightarrow{\mathcal{R}_k(\mathbf{x}_t)} v_{\text{left}} \quad \text{or} \quad v_k \xrightarrow{\neg \mathcal{R}_k(\mathbf{x}_t)} v_{\text{right}}. \quad (18)$$

The final prediction is the value at the leaf node: $\hat{y}_t = \mathbb{F}(\text{Leaf}(\mathbf{x}_t))$, where $\mathbb{F}$ is the value (e.g., majority class or mean) at the leaf.

Complexity: The computational complexity of training a Decision Tree depends on the depth of the tree and the number of features:

- **Training:** $O(k \cdot n \cdot \log n)$, where k is the number of features and n is the number of samples.
- **Inference:** $O(\log n)$, as traversing the tree requires a number of steps proportional to its depth.

3.4 Random Forest

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: The Random Forest algorithm uses an ensemble of trees T , where $|T|$ denotes the number of trees in the forest. Each is built using a random subset of the training data and a random set of features at each split.

For each tree, τ , the input $\{p_1, p_2, \dots, p_{t-1}\}$ yields an output $\hat{p}_t^{(\tau)}$. The output of the random forest is as follows:

$$\hat{p}_t = \frac{1}{|T|} \sum_{\tau \in T} \hat{p}_t^{(\tau)} \quad (19)$$

Identical to above, letting $\hat{\mathbf{p}} = \begin{bmatrix} \hat{p}_1 \\ \vdots \\ \hat{p}_n \end{bmatrix}$ and $\mathbf{p} = \hat{\mathbf{p}} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$, where n is the number of predictions we are making, we use MSE as the loss function:

$$L(\hat{\mathbf{p}}, \mathbf{p}) = \frac{1}{n} \sum_{t=1}^n (\hat{p}_t - p_t)^2 \quad (20)$$

Model Structure: For a Random Forest, the goal is to train several Decision Trees on random subsets of the dataset. Each tree produces its own prediction $\hat{y}_i$. The final prediction is typically taken to be the average of the predictions $\hat{y}_i$.

Training Process: The forest is constructed by training each tree on a random subset of the dataset. The entire forest is trained once each tree is trained.

Inference Process: For a new input vector $\mathbf{x}_t$, the prediction $\hat{y}$ is obtained by traversing each tree from root to a leaf and averaging the sub-predictions $\hat{y}_i$ given by each tree τ_i .

Complexity:

- **Training:** $O(T \cdot k \cdot n \cdot \log(n))$
- **Inference:** $O(T \cdot \log(n))$

4 Bi-directional RNNs

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: Sometimes, the predictions rely on both previous and future inputs such as in speech transcription, where a pronunciation maybe only made clear after a few words of context [Deep Learning by Goodfellow, Bengio, and Courville (2016)]. In this scenario, standard RNNs fail to capture the future dependencies. For this, we present bi-directional RNNs.

In this case, we have two hidden states, $\mathbf{h}^{(t)} \in \mathbb{R}^n$ and $\mathbf{g}^{(t)} \in \mathbb{R}^n$, where $\mathbf{h}^{(t)}$ propagates information forward in the sequence and $\mathbf{g}^{(t)}$ propagates information backwards.

$$\mathbf{a}_h^{(t)} = \mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b}_h, \quad \mathbf{h}^{(t)} = \sigma(\mathbf{a}_h^{(t)}) \quad (21)$$

$$\mathbf{a}_g^{(t)} = \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{W}_g \mathbf{g}^{(t+1)} + \mathbf{b}_g, \quad \mathbf{g}^{(t)} = \sigma(\mathbf{a}_g^{(t)}) \quad (22)$$

The output of the bi-directional RNN:

$$\mathbf{y}^{(t)} = \mathbf{W}_y \begin{bmatrix} \mathbf{h}^{(t)} \\ \mathbf{g}^{(t)} \end{bmatrix} + \mathbf{c} \quad (23)$$

Since we are predicting a single scalar value: the next closing price, we can write the following.

$$p_t = \mathbf{w}^T \begin{bmatrix} 1 \\ \mathbf{h}^{(t)} \\ \mathbf{g}^{(t)} \end{bmatrix} + \epsilon_t \quad (24)$$

where $\mathbf{w} = \begin{bmatrix} c \\ \phi_1 \\ \vdots \\ \phi_{2n} \end{bmatrix}$, where c is a bias term.

Each of the weight matrices and biases must be learned in this architecture.

Model Structure: For a bi-directional RNN, the goal is to use the preceding data, as well as the subsequent data, to make an accurate prediction. The model keeps two hidden-states: one for the previous data, and one for the future data. These hidden-states convey information to the model in order to make a more accurate prediction.

Training Process: For a given prediction, the gradient of the loss is computed with respect to each learnable weight matrix $\mathbf{W}$. Each weight matrix is updated based on an optimization algorithm, for example, gradient descent.

Inference Process: For a given input vector $\mathbf{x}_t$, the forward hidden state $\mathbf{h}^{(t)}$ captures information from the preceding k inputs, and the backward hidden state $\mathbf{g}^{(t)}$ captures information from the subsequent k inputs. These states are concatenated with a constant 1 to form the prediction vector:

$$\mathbf{z}^{(t)} = [1 \ \mathbf{h}^{(t)} \ \mathbf{g}^{(t)}]. \quad (25)$$

The prediction is then computed as:

$$p_t = \mathbf{w}^T \mathbf{z}^{(t)} + \epsilon_t, \quad (26)$$

where $\mathbf{w} \in \mathbb{R}^{2n+1}$ is the output weight vector, and ϵ_t is the error term.

Complexity:

- **Training:** $O(n \cdot t \cdot h^2)$, where n is the number of samples, t is the number of preceding and subsequent values, and h is the dimension of the hidden states.
- **Inference:** $O(t \cdot h^2)$

4.1 LSTMs

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: Long-short-term memory networks (LSTMs) are extensions of RNNs designed to overcome the problem of vanishing gradients during training. They include memory cells that help retain important information over longer periods, making them ideal for financial predictions where long-term dependencies matter.

Forget gate:

$$\mathbf{a}_f^{(t)} = \mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f, \quad \mathbf{f}^{(t)} = \sigma(\mathbf{a}_f^{(t)}) \quad (27)$$

where $\mathbf{W}_f$ and $\mathbf{U}_f$ are weight matrices and $\mathbf{b}_f$ is the forget gate bias.

Input gate:

$$\mathbf{a}_i^{(t)} = \mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i, \quad \mathbf{i}^{(t)} = \sigma(\mathbf{a}_i^{(t)}) \quad (28)$$

where $\mathbf{W}_i$ and $\mathbf{U}_i$ are weight matrices and $\mathbf{b}_i$ is the input gate bias.

Candidate cell state:

$$\mathbf{a}_c^{(t)} = \mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c, \quad \hat{\mathbf{c}}^{(t)} = \tanh(\mathbf{a}_c^{(t)}) \quad (29)$$

Cell state update:

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \hat{\mathbf{c}}^{(t)} \quad (30)$$

Output gate:

$$\mathbf{a}_o^{(t)} = \mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o, \quad \mathbf{o}^{(t)} = \sigma(\mathbf{a}_o^{(t)}) \quad (31)$$

where $\mathbf{W}_o$ and $\mathbf{U}_o$ are weight matrices and $\mathbf{b}_o$ is the output gate bias.

Hidden-state update:

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)}) \quad (32)$$

The output of the LSTM is as follows:

$$\mathbf{y}^{(t)} = \mathbf{W}_y \mathbf{h}^{(t)} + \mathbf{c} \quad (33)$$

Since we are predicting a single scalar value: the next closing price, we can write the following.

$$p_t = \mathbf{w}^T \begin{bmatrix} 1 \\ \mathbf{h}^{(t)} \end{bmatrix} + \epsilon_t \quad (34)$$

where $\mathbf{w} = \begin{bmatrix} c \\ \phi_1 \\ \vdots \\ \phi_n \end{bmatrix}$, with $\mathbf{h}^{(t)} \in \mathbb{R}^n$.

Model Structure: For an LSTM, the goal is to use the preceding data to make an accurate prediction. The main difference between the RNN and LSTM is the LSTM's ability to avoid the vanishing/exploding gradient problem that RNNs face. The LSTM accomplishes this via several states and gates: the cell-state, the hidden-state, the forget gate, the input gate, and the output gate.

Training Process: For a given prediction, the gradient of the loss is computed with respect to each learnable weight matrix $\mathbf{W}$. Each weight matrix is updated based on a given optimization method.

Inference Process: For a given prediction, the immediately preceding values are captured in the hidden-state, while the cell-state captures longer-range information. The forget-gate determines which information to discard from the previous cell-state. The input-gate determines which information to include in the new cell-state. The output-gate determines how much of the cell-state is included in the output.

Complexity:

- **Training:** $O(n \cdot t \cdot \text{epochs} \cdot (h^2 + h \cdot f))$, where n is the number of sequences, t is the number of time steps incorporated in each prediction, h is the dimension of the hidden-states, and f is the dimension of the input vectors.
- **Inference:** $O(h^2 + h \cdot f)$

4.2 Bi-directional LSTMs

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Forward Direction:

Forget gate:

$$\mathbf{f}_f^{(t)} = \sigma(\mathbf{W}_{f,f}\mathbf{h}^{(t-1)} + \mathbf{U}_{f,f}\mathbf{x}^{(t)} + \mathbf{b}_{f,f}) \quad (35)$$

where $\mathbf{W}_{f,f}$ and $\mathbf{U}_{f,f}$ are weight matrices and $\mathbf{b}_{f,f}$ is the forget gate bias.

Input gate:

$$\mathbf{i}_f^{(t)} = \sigma(\mathbf{W}_{i,f}\mathbf{h}^{(t-1)} + \mathbf{U}_{i,f}\mathbf{x}^{(t)} + \mathbf{b}_{i,f}) \quad (36)$$

where $\mathbf{W}_i$ and $\mathbf{U}_i$ are weight matrices and $\mathbf{b}_i$ is the input gate bias.

Candidate cell state:

$$\hat{\mathbf{c}}_f^{(t)} = \tanh(\mathbf{W}_{c,f}\mathbf{h}^{(t-1)} + \mathbf{U}_{c,f}\mathbf{x}^{(t)} + \mathbf{b}_{c,f}) \quad (37)$$

Cell state update:

$$\mathbf{c}_f^{(t)} = \mathbf{f}_f^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}_f^{(t)} \odot \hat{\mathbf{c}}_f^{(t)} \quad (38)$$

Output gate:

$$\mathbf{o}_f^{(t)} = \sigma(\mathbf{W}_{o,f}\mathbf{h}^{(t-1)} + \mathbf{U}_{o,f}\mathbf{x}^{(t)} + \mathbf{b}_{o,f}) \quad (39)$$

where $\mathbf{W}_o$ and $\mathbf{U}_o$ are weight matrices and $\mathbf{b}_o$ is the output gate bias.

Hidden-state update:

$$\mathbf{h}_f^{(t)} = \mathbf{o}_f^{(t)} \odot \tanh(\mathbf{c}_f^{(t)}) \quad (40)$$

Backward direction:

Forget gate:

$$\mathbf{f}_b^{(t)} = \sigma(\mathbf{W}_{f,b}\mathbf{h}^{(t-1)} + \mathbf{U}_{f,b}\mathbf{x}^{(t)} + \mathbf{b}_{f,b}) \quad (41)$$

where $\mathbf{W}_{f,b}$ and $\mathbf{U}_{f,b}$ are weight matrices and $\mathbf{b}_{f,b}$ is the forget-gate bias.

Input gate:

$$\mathbf{i}_b^{(t)} = \sigma(\mathbf{W}_{i,b}\mathbf{h}^{(t-1)} + \mathbf{U}_{i,b}\mathbf{x}^{(t)} + \mathbf{b}_{i,b}) \quad (42)$$

where $\mathbf{W}_{i,b}$ and $\mathbf{U}_{i,b}$ are weight matrices and $\mathbf{b}_{i,b}$ is the input gate bias.

Candidate cell state:

$$\hat{\mathbf{c}}_b^{(t)} = \tanh(\mathbf{W}_{c,b}\mathbf{h}^{(t-1)} + \mathbf{U}_{c,b}\mathbf{x}^{(t)} + \mathbf{b}_{c,b}) \quad (43)$$

Cell state update:

$$\mathbf{c}_b^{(t)} = \mathbf{f}_b^{(t)} \odot \mathbf{c}_b^{(t-1)} + \mathbf{i}_b^{(t)} \odot \hat{\mathbf{c}}_b^{(t)} \quad (44)$$

Output gate:

$$\mathbf{o}_b^{(t)} = \sigma(\mathbf{W}_{o,b}\mathbf{h}^{(t-1)} + \mathbf{U}_{o,b}\mathbf{x}^{(t)} + \mathbf{b}_{o,b}) \quad (45)$$

where $\mathbf{W}_{o,b}$ and $\mathbf{U}_{o,b}$ are weight matrices and $\mathbf{b}_{o,b}$ is the output gate bias.

Hidden-state update:

$$\mathbf{h}_b^{(t)} = \mathbf{o}_b^{(t)} \odot \tanh(\mathbf{c}_b^{(t)}) \quad (46)$$

Hidden-state concatenation:

$$\mathbf{h}^{(t)} = \begin{bmatrix} \mathbf{h}_f^{(t)} \\ \mathbf{h}_b^{(t)} \end{bmatrix} \in \mathbb{R}^{2n} \quad (47)$$

The output of the LSTM is as follows:

$$\mathbf{y}^{(t)} = \mathbf{W}_y \mathbf{h}^{(t)} + \mathbf{c} \quad (48)$$

Since we are predicting a single scalar value: the next closing price, we can write the following.

$$p_t = \mathbf{w}^T \begin{bmatrix} 1 \\ \mathbf{h}^{(t)} \end{bmatrix} + \epsilon_t \quad (49)$$

where $\mathbf{w} = \begin{bmatrix} c \\ \phi_1 \\ \vdots \\ \phi_{2n} \end{bmatrix}$.

Model Structure: For a bi-directional LSTM, the goal is to use the preceding data and the subsequent data to make an accurate prediction. Each direction (forward and backward) has its own set of gates and states as detailed in the LSTM section.

Training Process: For a given prediction, the gradient of the loss is computed with respect to each learnable weight matrix $\mathbf{W}$. Each weight matrix is updated based on a given optimization method.

Inference Process: For a given prediction, the immediately preceding values are captured in the forward hidden-state, the immediately subsequent values are captured in the backward hidden-state. The cell-states capture the forward and backward longer-term memory. The gates function identically to their standard LSTM counterparts.

Complexity:

- **Training:** $O(n \cdot t \cdot \text{epochs} \cdot (h^2 + h \cdot f))$, where n is the number of sequences, t is the number of time steps incorporated in each prediction, h is the dimension of the hidden-states, and f is the dimension of the input vectors.
- **Inference:** $O(h^2 + h \cdot f)$

5 Transformers

Input: A matrix containing $t - 1$ vectors $\mathbf{x}_i$, where each is the same arbitrary length.

Output: A prediction given an input vector $\mathbf{x}_i$.

Objective Function: Transformers have recently become popular for time-series prediction due to their ability to model long-range dependencies using a self-attention mechanism. Unlike RNNs and LSTMs, transformers do not process data sequentially, making them more efficient for large datasets and allowing them to capture complex, long-term relationships in stock market data.

In order to use a transformer on time series data $\{p_1, \dots, p_t\}$, we first embed each p_t into a higher-dimensional vector. Given our sequence of embeddings $\{\mathbf{x}_1, \dots, \mathbf{x}_t\}$, we now add a positional embedding to each.

$$\mathbf{z}_t = \mathbf{x}_t + \mathbf{p}(t) \quad (50)$$

This sequence of embeddings can be represented as an embedding matrix $\mathbf{Z}$.

$$\mathbf{Z} = [\mathbf{z}_1 \quad \dots \quad \mathbf{z}_t] \quad (51)$$

The mechanism that makes the transformer architecture so powerful is the attention block. Self-attention is the action of computing attention weights that determine the importance of each element in the sequence.

For a given embedding $\mathbf{Z}$, we compute the query matrix $\mathbf{Q}$.

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q \quad (52)$$

where $\mathbf{W}^Q$ is a learned matrix. Each row in $\mathbf{Q}$ corresponds to a query for one token in the sequence.

The key matrix, $\mathbf{K}$, is computed similarly.

$$\mathbf{K} = \mathbf{Z}\mathbf{W}^K \quad (53)$$

where $\mathbf{W}^K$ is a learned matrix. Each row in $\mathbf{K}$ can be interpreted as representing the information offered by the corresponding token.

The attention weights are computed with $\mathbf{Q}$ and $\mathbf{K}$.

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \quad (54)$$

where d_k is the key dimension.

The value matrix, $\mathbf{V}$, is also computed in a similar way to the query and the key matrices.

$$\mathbf{V} = \mathbf{Z}\mathbf{W}^V \quad (55)$$

where $\mathbf{W}^V$ is a learned matrix. Each row in $\mathbf{V}$ represents the information that is available for use in the output.

The output of the attention layer, $\mathbf{O}$, is computed with $\mathbf{A}$ and $\mathbf{V}$.

$$\mathbf{O} = \mathbf{A}\mathbf{V} \quad (56)$$

After the attention layer, there are usually some linear layers (feedforward networks).

The output of the transformer is usually given by a vector in the embedding space. For our purpose, we can use a linear layer to project this down to a scalar, a closing price.

The modularity of the transformer architecture allows for several modifications. One can introduce hidden states, such as in the RNN or LSTM, to 'augment' the attention mechanism with a memory. This serves to reduce the computational complexity that is associated with large context windows. Other modifications, including the convolutional network, have been explored as well.

Model Structure: For the transformer, the goal is to use the preceding data to make an accurate prediction. The key feature of the transformer, the attention block, serves to 'measure' the importance

of each data point to each other. For this reason, it is able to make inferences with long-term dependencies with ease.

Training Process: First, the transformer embeds the data into a higher dimensional space. Then, positional encodings are added to each data point to give the transformer the ordering of the data. From here, the query, key, and value matrices are calculated and the attention calculation is performed. Each position undergoes a two-layer feed-forward network. The output is given by a layer norm of the resulting values. The loss is computed with respect to each learnable matrix $\mathbf{W}$. Each weight matrix is updated based on the given optimization method, typically Adam.

Inference Process: Exactly like the training process, the attention calculation is performed and the output is given. The output is then decoded, bringing it from its embedding dimension down to a scalar value $\hat{y}$.

Complexity:

- **Training:** $O(n \cdot \text{epochs} \cdot L \cdot (t^2 \cdot d + t \cdot d^2))$ where n is the batch-size, L is the number of attention layers, t is the number of 'tokens' including in the prediction, and d is the embedding dimension.
- **Inference:** $O(L \cdot (t^2 \cdot d + t \cdot d^2))$

6 Research Summary

In this research, different autoregressive machine-learning models were experimented with to approximate an unknown quantum wave (target function). Initially, simple regression models such as linear regression and decision trees were utilized. A simple neural network was developed, and Large Language Model (LLM) architectures were also experimented with to approximate the target function. The simpler models served as a baseline for comparison with the LLMs. LLMs are expected to outperform all other models, as they are specifically engineered for time series data. At every stage, the same data preparation steps were applied with the exception of experimentation with features.

The first model used was the standard linear regression model. This model serves as a simple starting point for machine learning beginners. Linear regression was applied to cleaned and smoothed closing price data. To smooth the data, two paradigms were used: moving averages and Gaussian pulse transformations. The results were documented in Github. The next model used was the ARMA model. This model is similar to the linear regression model but incorporates its errors into its predictions. The ARMA model works best with stationary data, so differencing was applied to achieve stationarity.

Deep-learning architectures were also used to model the market as a wave function. These were neural networks and RNNs. Neural networks provided a highly complex model that The motivation behind using an RNN was the potential to capture non-local dependencies in the data, a very common feature of financial data. The results were documented in Github.

The main focus of these models is to explore the possibilities of quantum-inspired predictions and teach students about the various applications of regression models. The universality of the financial market, as well as the potential reward for modeling it with high accuracy, makes for an excellent learning example. In each model's file, it is shown how to instantiate the model, how to fit/train it, and evaluation techniques, including visualizations of the model's performance. Where applicable, the hyperparameters of a model are discussed and the students are encouraged to tune them to achieve higher fidelity predictions.

This document functions, primarily, as a reference for the implementations of various autoregressive models. The mathematical and algorithmic descriptions capture the essence of the models, while the modules in the Github provide students with the tools and knowledge to implement their own models. Together, the modules and this document serve as an excellent reference for implementing well-studied machine-learning models. After reviewing the modules, students should be able to demonstrate a fundamental understanding of each model as well as have the tools to implement their own in a unique context. Additionally, students should know how to determine the model to use based on the nature of the target function and the data quantity/quality.

7 Conclusion and Future Work

This study has analyzed the prediction capabilities of different autoregressive models on stock price data. The details and mathematics behind each model are given and consolidated into this document, which can be used as a reference for implementing them. Code alongside the mathematical formalization was developed to demonstrate the practical applications of this work. The main task was to use these models to approximate the quantum wave target function, and that was done by performing feature engineering that added wave features to the input data.

7.1 Key Takeways

- A proper balance of model complexity and feature engineering is necessary to obtain good predictions.
- Most machine learning (ML) models are variations, extensions, or combinations of simpler models.

7.2 Implications For Machine Learning Beginners

- It is good practice to perform feature engineering before moving on to more complex models.
- Understanding the mathematics behind simpler ML models is crucial to understanding more complex ones.
- Each implementation should be compared to implementations from reputable online sources, and then deeply fine-tuned following the addition of new features.
- Focus on only a few models until using a new model becomes necessary due to complexity or type of data (LLM's are promising in this case due to their excellent performance on sequential data).
- Have well-defined and simple goals that increase in complexity.

7.3 Future Work

Future research will focus more on feature engineering wave-like data and utilizing pre-trained LLM's. This method allows for experimentation that does not deviate much from reputable baseline implementations. Future research should also focus on the fine details of engineering a quantum wave target function. These improvements will help the research obtain more significant results by reducing the uncertainty within the experimentation. Additionally, researchers should communicate with domain experts as frequently as possible to avoid spending valuable time on unnecessary pursuits.

8 Appendix

8.1 Neural Networks

A neural network consists of layers of interconnected nodes, which are usually referred to as neurons. Each neuron takes in an input, applies a linear transformation, and then passes the result into a non-linear 'activation' function. During training, the linear transformations are adjusted to better map the inputs to the outputs.

A single neuron takes an input vector $\mathbf{x}$ and applies a transformation as follows:

$$z = \mathbf{w}^T \mathbf{x} + b \quad (57)$$

where $\mathbf{w}$ is a vector of weights and b is the bias term. The non-linear activation function is applied as follows:

$$a = \sigma(z) \quad (58)$$

where σ is the activation and a is the output of the neuron.

Neural networks are composed of several layers. There is the input layer which represents the input features. In our case, it will be the last k closing prices. Then, there are hidden layers. These layers are where the network of neurons processes inputs through linear transformations and non-linear activations. Lastly, there is an output layer. This layer produces the final prediction of the model; in our case, this will be a single neuron which predicts the next closing price.

8.2 Activations

Activation functions introduce non-linearity to the model, facilitating complex learning. The most common activation functions are ReLU, sigmoid, and tanh.

ReLU or Rectified Linear Unit is the most simple and it is widely used because it is able to mitigate the problem of vanishing gradients.

$$\sigma(z) = \max\{0, z\} \quad (59)$$

Sigmoid activation function maps the input to a range between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (60)$$

Tanh activation function maps the input to the range between -1 and 1.

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (61)$$

8.3 Backpropagation and Gradient Descent

To train the model, we perform an algorithm called backpropagation. We compute the gradient of the loss function with respect to the weights. The weights are then updated using gradient descent, which aims to find minima of the loss function.

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

where η is the learning rate and $\frac{\partial L}{\partial W^{(l)}}$ is the gradient of the loss with respect to the weights at layer l .

8.4 RNNs

Recurrent Neural Networks (RNNs) are a type of neural network particularly well-suited for time series data because they have connections that form cycles, allowing them to "remember" information from previous time steps. This makes them effective in capturing sequential dependencies.

The key feature of RNNs is that they have a time-dependent hidden state. The prediction at each time step depends on both the input at the time step, as well as the hidden state at that time step. The hidden state is updated:

$$\mathbf{a}^{(t)} = \mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b} \quad (62)$$

$$\mathbf{h}^{(t)} = \sigma(\mathbf{a}^{(t)}) \quad (63)$$

where $\mathbf{h}^{(t)}$ is the hidden state at time t , $\mathbf{x}^{(t)}$ is the input at time t , $\mathbf{W}_h$ is the weight matrix for the hidden state, $\mathbf{W}_x$ is the weight matrix for the input, $\mathbf{b}$ is the bias vector, and σ is the activation function, typically tanh or ReLU.

The output of the RNN at each time step is given by:

$$\mathbf{y}^{(t)} = \mathbf{W}_y \mathbf{h}^{(t)} + \mathbf{c} \quad (64)$$

where $\mathbf{W}_y$ is the weight matrix of the output and $\mathbf{c}$ is the output bias vector. In our case, we are predicting the next closing price given a sequence of closing prices. Our hidden state remains a vector, however, our output weight matrix is just a vector and the output bias is just a scalar.

In order to train the model, we must modify our backpropagation algorithm. This modified version is called Backpropagation Through Time (BPTT).

Output layer gradients:

$$\delta_y^{(t)} = \frac{\partial L^{(t)}}{\partial \mathbf{y}^{(t)}} \quad (65)$$

$$\frac{\partial L}{\partial \mathbf{W}_y} = \sum_{t=1}^k \delta_y^{(t)} \left(\mathbf{h}^{(t)} \right)^T \quad (66)$$

$$\frac{\partial L}{\partial \mathbf{c}} = \sum_{t=1}^k \delta_y^{(t)} \quad (67)$$

Hidden-layer gradients:

$$\delta_h^{(k)} = \left(\mathbf{W}_y^T \delta_y^{(k)} \right) \odot \sigma'(\mathbf{a}^{(k)}) \quad (68)$$

where σ' denotes the derivative of σ with respect to $\mathbf{a}^{(t)}$ and $\odot$ denotes element-wise multiplication.

$$\delta_h^{(t)} = \left(\mathbf{W}_y^T \delta_y^{(t)} + \mathbf{W}_h^T \delta_h^{(t+1)} \right) \odot \sigma'(\mathbf{a}^{(t)}) \quad (69)$$

$$\frac{\partial L}{\partial \mathbf{W}_h} = \sum_{t=1}^k \delta_h^{(t)} \left(\mathbf{h}^{(t-1)} \right)^T \quad (70)$$

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{t=1}^k \delta_h^{(t)} \quad (71)$$

where $\mathbf{h}^{(0)} = \mathbf{0}$.

Input layer gradients:

$$\frac{\partial L}{\partial \mathbf{W}_x} = \sum_{t=1}^k \delta_h^{(t)} \left(\mathbf{x}^{(t)} \right)^T \quad (72)$$

RNNs suffer from a phenomenon known as vanishing and exploding gradients [Deep Learning by Goodfellow, Bengio, and Courville (2016)]. When $\|\mathbf{W}_h^T\| < 1$, repeated multiplication can cause the gradients to tend towards zero, making long-term dependencies difficult to learn. Similarly, when $\|\mathbf{W}_h^T\| > 1$, gradients can grow exponentially, causing instability in the weights. To combat this, BPTT is sometimes truncated to a shorter, fixed sequence length.

9 References

- [1] Melko, R.G., Carrasquilla, J. Language models for quantum simulation. *Nat Comput Sci* 4, 11-18 (2024). <https://doi.org/10.1038/s43588-023-00578-0>
- [2] Wong, Thomas. *Introduction to Classical and Quantum Computing*. Rooted Grove, 2022.
- [3] Lipton, Richard J, and Kenneth W Regan. *Introduction to Quantum Algorithms via Linear Algebra*. The MIT Press, 2021.
- [4] Kamath, Uday, et al. *Transformers for Machine Learning*. CRC Press, 2022.
- [5] Vaswani, Ashish, et al. Attention Is All You Need. 2 Aug. 2023, arxiv.org/abs/1706.03762.
- [6] Wilde, Mark M. *Quantum Information Theory*. Cambridge University Press, 2017.
- [7] "Yahoo Finance - Stock Market Live, Quotes, Business & Finance News." Yahoo! Finance, Yahoo!, finance.yahoo.com/. Accessed 21 Aug. 2024.
- [8] Abu-Mostafa, Magdon-Ismail, Lin, *Machine Learning From Data*, 2012
- [9] Ivanov, Radoslav. "Reinforcement Learning." *CSCI 4160/6963*, Fall 2024, <http://www.cs.rpi.edu/~ivanor/rl/F24/rl.html>. Accessed 14 Dec. 2024.
- [10] Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory." *Neural Computation*.
- [11] Graves, A., & Schmidhuber, J. (2005). "Framewise phoneme classification with bidirectional LSTM and other neural network architectures." *Neural Networks*.
- [12] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [13] "Find and Share Research." ResearchGate, www.researchgate.net/. Accessed 14 Dec. 2024.
- [14] "Towards Data Science." Towards Data Science, towardsdatascience.com/. Accessed 14 Dec. 2024.